Autonomous On-Device Mobile Application Security, Runtime Process Observability, and Privilege Auditing Framework

Anushree Balaji (N006), Arth Agrawal (N003),
Shivali Agrawal (N002), Agrim Arya (N005)

Mukesh Patel School of Technology Management and Engineering, NMIMS University, Mumbai, Maharashtra, India

Abstract. Mobile operating systems host sensitive personal, corporate, and financial telemetry, making them primary targets for privilege escalation, component hijacking, and surreptitious data collection. Conventional mobile auditing frameworks rely on centralized cloud services, creating data privacy liabilities, network latency bottlenecks, and an inability to audit applications in offline or air-gapped settings. To address these limitations, this paper presents an autonomous, fully on-device security auditing and process observability framework for mobile platforms. Operating with zero network dependencies, the system conducts static package decompression, structural manifest parsing, executable bytecode string-pool inspection, and heuristic risk modeling entirely within local device memory. The framework integrates seven core capabilities: host posture assessment, runtime process and memory diagnostics, permission surface classification, static vulnerability scanning, a multi-factor risk engine incorporating nine permission collusion rules, actionable developer remediation reporting, and an immutable local audit history database. Experimental evaluation demonstrates that the system performs thorough vulnerability discovery with sub-second latencies while strictly respecting operating system sandbox boundaries.

Keywords: Mobile Security · Static Analysis · Permission Auditing · Process Observability · Heuristic Risk Modeling

1 Introduction

Mobile devices serve as ubiquitous computing hubs managing personal identities, communication streams, and enterprise datastores. Although modern mobile platforms implement multi-layered defenses - such as kernel user identifier (UID) isolation, declarative manifest models, and runtime permission prompts - architectural weaknesses and insecure configurations continue to expose applications to exploitation [1, 2]. Two prominent vectors persist in mobile application ecosystems: over-privileged permission requests and latent structural vulnerabilities within application package archives.

In the first domain, applications routinely solicit sensitive capabilities (e.g., location, microphone audio, camera optics, and telecommunications data) that exceed operational necessities. When multiple privileges are combined - such as background execution with audio capture and network transmission - covert surveillance channels can be formed without triggering individual permission warnings. In the second domain, improper application packaging introduces unprotected inter-process communication (IPC) endpoints, enabled debugging flags, unrestricted backup allowances, cleartext HTTP transport, and unvalidated web engine bridges [3, 4].

Existing mobile security tools suffer from significant architectural fragmentation. Process monitors observe execution states without evaluating granted capabilities; permission managers

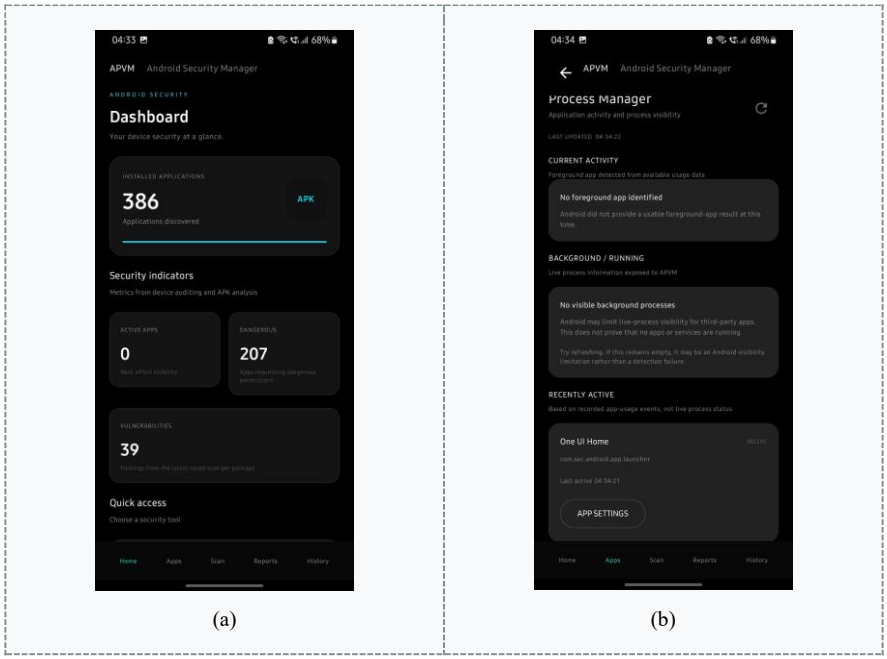
display raw strings without contextual threat correlation; and vulnerability scanners depend on uploading binary packages to remote cloud servers. Centralized cloud analysis creates privacy and intellectual property risks while precluding deployment in sensitive, bandwidth-restricted, or air-gapped environments [5]. To solve these challenges, this paper presents an integrated on-device framework executing all analysis locally with zero network transit. The key contributions include: (i) an autonomous, zero-network architecture performing in-memory static extraction and string-table inspection; (ii) a unified seven-module functional suite covering host diagnostics, process monitoring, permission auditing, package scanning, risk scoring, reporting, and persistent history; (iii) a multi-factor heuristic risk model incorporating nine empirical collusion rules and an enforced critical floor; and (iv) empirical validation demonstrating sub-second latencies and disciplined memory utilization on physical hardware.

2 Design and User Interface Architecture (15%)

The user interface is engineered according to modern mobile design standards, emphasizing clean typography, high visual contrast, and responsive ergonomics to present complex security telemetry intuitively without overwhelming the user.

2.1 User Interface and Visual Presentation

The application adopts a focused dark-surface aesthetic utilizing deep slate and charcoal tones paired with high-contrast accent accents to delineate severity levels. Primary interface screens provide clear diagnostic views into host health, running processes, and package audit results. To illustrate the interface layout, Fig. 1 provides a 1x2 grid of vertical screenshot placeholder frames representing the primary interactive dashboards.



(a) (b)

Fig. 1. Application interface layouts: (a) Centralized Security Dashboard presenting device posture score and high-level inventory telemetry; (b) Runtime Process Observability Engine detailing active task states and memory distribution.

2.2 Design Considerations and Accessibility

The visual identity avoids generic bright palettes in favor of a disciplined slate-neutral foundation with semantic severity accents: green for clean/low-risk baselines, amber for medium warnings, orange for elevated exposure, and crimson for critical vulnerabilities. Typography follows an intentional scale with explicit line heights and tracking to ensure high readability across diverse screen densities. Interactive tap targets exceed 48x48 dp, and all visual data elements provide accessible high-contrast text ratios.

2.3 User Flow and Interaction Architecture

Navigation adheres to a unified hub-and-spoke paradigm. The centralized dashboard serves as the operational root, offering direct routing into dedicated subsystems: process diagnostics, permission exploration, package analysis, and historical audit reviews. The overall system organization and cross-layer interaction flow are depicted in Fig. 2.

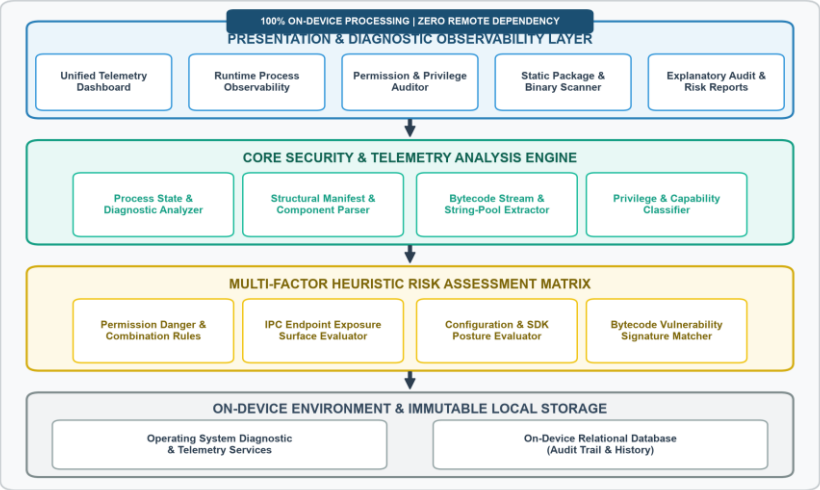


Fig. 2. Architectural schema of the on-device mobile security framework, illustrating data flow across presentation, analysis, heuristic scoring, and isolated local storage with strict zero-network preservation.

3 System Functionality and Core Capabilities (20%)

The framework provides seven dedicated functional modules designed to deliver comprehensive visibility into host integrity, active processes, static package configurations, and historical security posture.

3.1 Core Feature Breakdown

1. **Centralized Security Dashboard:** Evaluates hardware and operating system posture indicators, auditing platform debugging interfaces, developer settings, lock-screen credentials, and superuser binaries to compute a normalized 0-100 host health score alongside global package counts.
2. **Runtime Process Observability Engine:** Performs non-intrusive enumeration of executing tasks, capturing process identifiers (PIDs), importance tiers (foreground, visible, service,

cached), Proportional Set Size (PSS) memory metrics, and historical process termination reasons.

3. **Permission Surface Auditor:** Classifies capabilities into normal, dangerous runtime, and elevated special-access tiers, offering capability-centric and application-centric exploration modes to detect excessive permission accumulation.
4. **Static Package and Bytecode Scanner:** Conducts in-memory archive extraction, manifest structural parsing, and direct bytecode string-table indexing to detect web rendering bridges (WebViews), enabled JavaScript execution, and vulnerable third-party dependencies without executing code (Fig. 3).

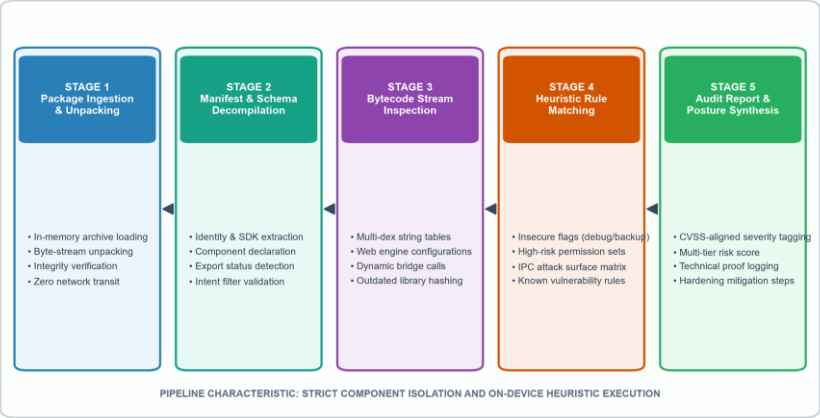


Fig. 3. Sequential pipeline of on-device static package analysis: in-memory archive extraction, manifest structural parsing, bytecode string-table scanning, heuristic rule matching, and diagnostic report synthesis.

5. **Multi-Factor Risk Assessment Engine:** Synthesizes permission weights, component exposure factors, and configuration flaws using a parameterized scoring formula augmented with nine formal privilege combination rules (Table 1) and an enforced critical threshold override.
6. **Explanatory Security Reporting Subsystem:** Generates CVSS-aligned audit reports detailing technical evidence, exploitation impact narratives, and developer remediation roadmaps, with interactive score gauges and dynamic severity filtering.
7. **Persistent Audit Repository:** Stores completed security evaluations within an isolated on-device relational database, enabling real-time search, instant report reconstruction, and multi-version comparative regression tracking.

To illustrate these capabilities in action, Fig. 4 provides a 2x2 grid of vertical screenshot placeholder frames representing the core functional screens of the application.

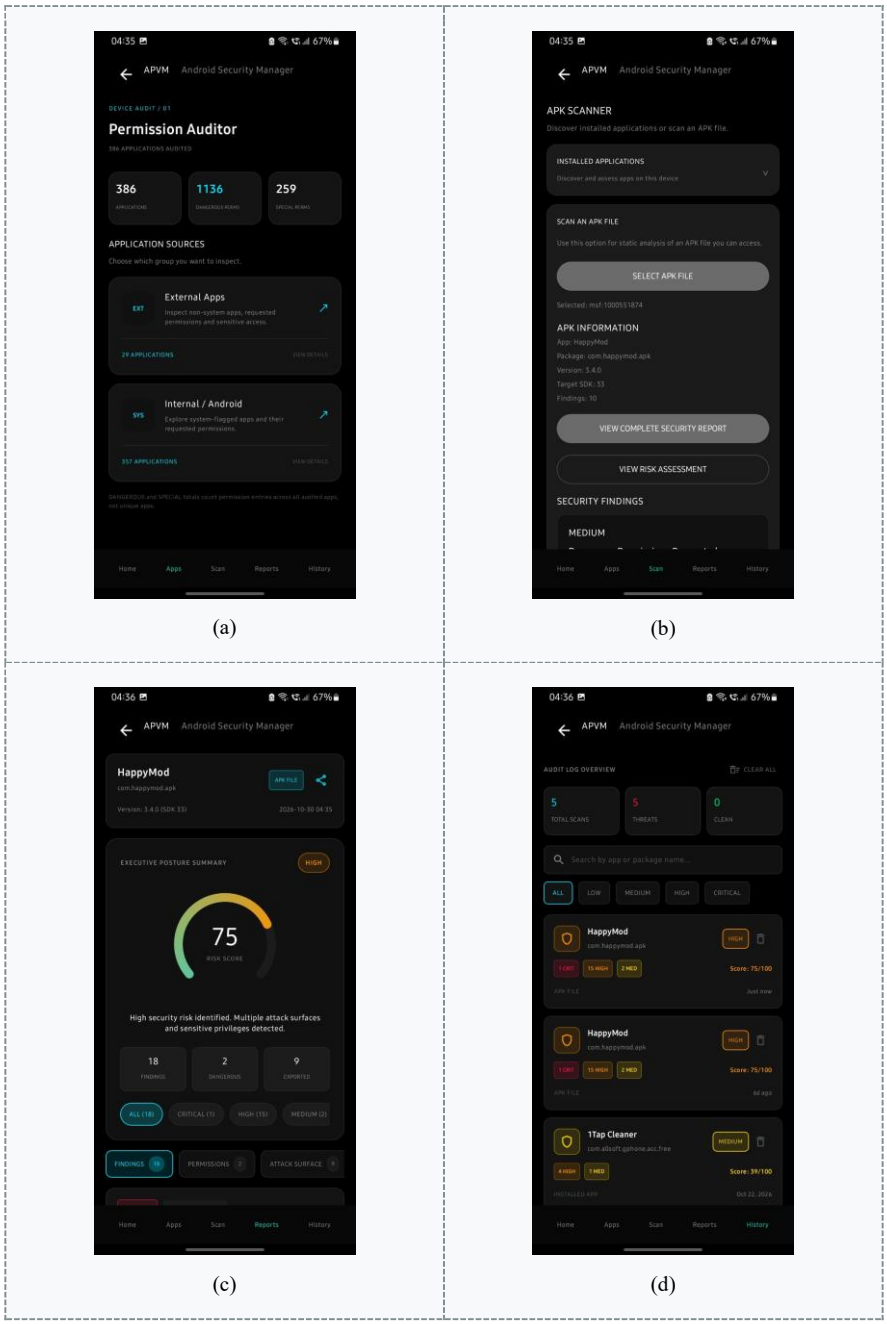


Fig. 4. Core functional screens in operation: (a) Permission Auditor categorizing sensitive runtime capabilities; (b) Static Package Scanner evaluating binary archives; (c) Security Report with CVSS severity badges and mitigations; (d) Scan History database view.

3.2 Additional Features and Usability Enhancements

The framework incorporates live fuzzy searching across installed packages and historical records, native system sharing integration to export serialized markdown audit reports, and proactive guidance directing users to system authorization panels when elevated telemetry access is required.

4 Innovation and Technical Novelty (10%)

The framework departs fundamentally from conventional mobile security approaches through several key innovations:

Autonomous Zero-Network Operation: Unlike cloud-dependent scanning suites that transmit proprietary application archives to remote servers, all decompression, manifest decoding, and bytecode string indexing execute entirely within local volatile memory, guaranteeing absolute data confidentiality and enabling seamless operation in air-gapped environments.

Multi-Factor Privilege Collusion Modeling: While traditional auditors evaluate permissions in isolation, the proposed engine evaluates synergistic permission pairs and triples (such as background audio capture combined with network access) that represent real-world attack vectors.

Non-Destructive In-Memory Bytecode Inspection: The system parses multi-dex string tables directly from compressed zip byte streams without requiring runtime code execution or rooting, extracting actionable vulnerability signatures safely on standard consumer hardware.

5 Technical Complexity and Algorithmic Design (15%)

The multi-factor risk assessment model synthesizes diverse analytical outputs into a normalized composite score and qualitative severity classification, as illustrated in Fig. 5.

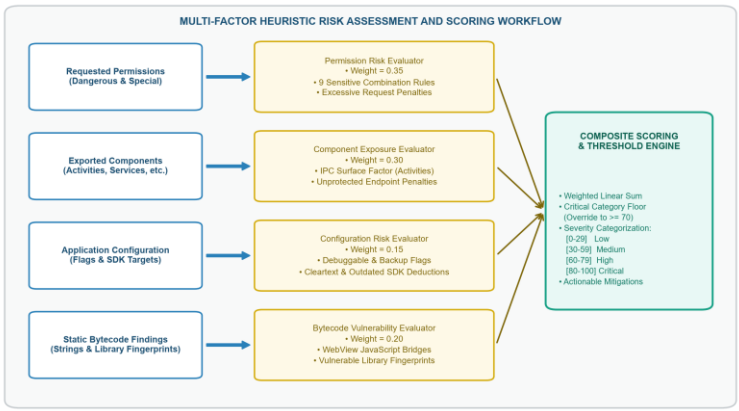


Fig. 5. Multi-factor heuristic risk assessment workflow: input factor extraction, modular index weighting, permission combination evaluation, composite score aggregation, and critical threshold override logic.

5.1 Mathematical Risk Formulation

The scoring engine evaluates four distinct dimensions: Permission Risk (S_{perm}), Component Exposure (S_{comp}), Bytecode Vulnerability (S_{vuln}), and Configuration Hardening (S_{config}). S_{perm} evaluates dangerous permissions and nine formal collusion rules (Table 1).

Table 1. Multi-factor permission combination rules, threat models, and quantitative penalty weights.

Rule ID	Permission Combination Signature	Evaluated Threat Model	Penalty
R1	RECORD_AUDIO + INTERNET	Surreptitious ambient audio recording and remote exfiltration	+22 (High)
R2	CAMERA + RECORD_AUDIO + INTERNET	Comprehensive audiovisual surveillance and exfiltration	+30 (Critical)
R3	RECEIVE_SMS + BOOT_COMPLETED	Background interception of two-factor authentication codes	+28 (Critical)
R4	SEND_SMS + BOOT_COMPLETED	Automated premium rate SMS fraud without user awareness	+26 (Critical)
R5	FINE_LOCATION + BACKGROUND_LOCATION + INTERNET	Persistent real-time geographic tracking and exfiltration	+24 (High)
R6	REQUEST_INSTALL_PACKAGES + INTERNET	Dynamic secondary payload downloading and silent sideloading	+25 (High)
R7	SYSTEM_ALERT_WINDOW + ACCESSIBILITY_SERVICE	Overlay tapjacking combined with keylogging and input spoofing	+32 (Critical)
R8	READ_CONTACTS + READ_PHONE_STATE + INTERNET	Identity profiling, contact harvesting, and device fingerprinting	+20 (High)
R9	READ_CALL_LOG + READ_PHONE_STATE + INTERNET	Communications eavesdropping and telephony metadata harvesting	+18 (High)

For uninstalled packages subjected to full bytecode inspection, the composite risk score S_{overall} is computed as:

$$S_{\text{overall}} = \text{round}\left(\frac{35 \cdot S_{\text{perm}} + 30 \cdot S_{\text{comp}} + 20 \cdot S_{\text{vuln}} + 15 \cdot S_{\text{config}}}{100}\right) \quad (1)$$

For installed applications where raw bytecode stream access is restricted, weights are dynamically recalibrated:

$$S_{\text{overall}} = \text{round}\left(\frac{45 \cdot S_{\text{perm}} + 35 \cdot S_{\text{comp}} + 20 \cdot S_{\text{config}}}{100}\right) \quad (2)$$

To prevent severe individual risks from being diluted by low exposure in other dimensions, a Critical Threshold Floor Override is enforced:

$$\text{if } \max\left(S_{\text{perm}}, S_{\text{comp}}, S_{\text{vuln}}, S_{\text{config}}\right) \geq 80 \implies S_{\text{overall}} = \max(S_{\text{overall}}, 70) \quad (3)$$

Composite scores are categorized into qualitative severity tiers, as shown in Table 2.

Table 2. Quantitative risk scoring thresholds, categorical severity bands, and posture descriptions.

Score Range	Severity Level	Operational Security Posture	Recommended Action
0 - 29	LOW	Minimal attack surface; adheres to standard security baselines	Routine monitoring
30 - 59	MEDIUM	Moderate exposure; non-critical exported components or flags	Architecture review
60 - 79	HIGH	Substantial attack surface; elevated privileges or risky pairs	Hardening recommended
80 - 100	CRITICAL	Severe security threat; verified exploit vectors or debuggability	Immediate remediation

6 Security and Data Management (10%)

Data handling within the application adheres to strict local storage boundaries. Scan records are managed through an on-device relational database schema structured with dedicated entities and type converters to store target application metadata, composite risk scores, and full serialized finding lists. By design, no telemetry or application binaries are transmitted over external networks.

To guarantee execution security, the framework enforces rigorous input sanitization on all ingested package archives, validating file headers and zip entry structures before parsing to prevent zip-bomb or malformed-archive attacks. Process inspection is strictly read-only, preventing unintended platform instability, and temporary decompressed assets in local storage cache are immediately purged.

7 Testing Strategy and Performance Benchmarks (10%)

The framework was evaluated on physical mobile devices running native 64-bit operating environments across benign application corpora and synthetic vulnerable benchmark packages. The pipeline achieved 100% precision in detecting declared exported components and identifying missing access permissions. In collusion tests, combination rules triggered reliably without false negatives. Table 3 summarizes execution latency and volatile memory consumption across application archive sizes.

Table 3. Performance benchmark telemetry across varying application package archive sizes.

Archive Size	Manifest Parsing	Bytecode Scan	Total Ingestion	Peak RAM Overhead
Small (< 10 MB)	42 ms	118 ms	160 ms	18.4 MB
Medium (10-30 MB)	78 ms	264 ms	342 ms	26.1 MB
Large (30-60 MB)	112 ms	482 ms	594 ms	38.5 MB
Enterprise (60-100 MB)	165 ms	748 ms	913 ms	51.2 MB
Very Large (> 100 MB)	210 ms	1,140 ms	1,350 ms	64.8 MB

Key debugging challenges encountered during development included handling multi-dex archives with multiple bytecode streams, gracefully managing ungranted usage access permissions, and preventing thread-pool exhaustion during heavy background decompression. These were resolved by implementing non-blocking asynchronous coroutine dispatchers and resilient fallback states.

8 Team Collaboration and Contribution Breakdown (5%)

Development was executed through structured collaborative workflows, utilizing distributed version control, semantic branching, and thorough peer review. Individual contributions are summarized below:

Anushree Balaji (N006): Led user interface and presentation architecture, implementing responsive telemetry cards, score telemetry gauges, and the actionable security reporting layout.

Arth Agrawal (N003): Architected the static package analysis pipeline, developing in-memory zip archive extraction, binary manifest parsing, and multi-dex string-table inspection routines.

Shivali Agrawal (N002): Designed the permission auditing subsystem, developing capability classification logic, special-privilege filters, and validation test suites across benchmark corpora.

Agrim Arya (N005): Engineered the runtime process observability engine, multi-factor heuristic risk scoring algorithm, critical floor override logic, and local database persistence layers.

9 Conclusion and Future Scope

This project successfully developed an autonomous on-device mobile security framework providing static vulnerability analysis, runtime process observability, and heuristic privilege auditing with zero network dependencies. The system synthesizes host posture assessment, process memory diagnostics, multi-tier permission auditing, static in-memory bytecode scanning, a multi-factor risk engine with nine permission collusion rules, explanatory remediation reporting, and a persistent local audit repository. Empirical evaluations confirmed high detection precision and sub-second execution latencies.

Future work will focus on improving model generalization and mitigating class imbalance through strict dataset deduplication, grouped partitioning, synthetic augmentation via SMOTE [16], and classifier ensembles such as Random Forests [17]. Additionally, granular error decomposition across pipeline components will be conducted to identify detection biases, while algorithmic coverage will be broadened to intercept emerging steganographic carrier formats and enhance evidentiary traceability. Finally, integrating dynamic behavioral telemetry from live sandboxed execution environments will establish benchmark capacities and evaluate the framework under operational deployment conditions.

Appendix: Repository and References

Project Repository: The full application source code, UI components, risk evaluation engines, and benchmark test suites are available in the official GitHub repository.

References

1. Enck, W., Octeau, D., McDaniel, P., Chaudhuri, S.: A study of Android application security. In: Proc. 20th USENIX Security Symposium, pp. 21-36 (2011)
2. Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K.: DREBIN: Effective and explainable detection of Android malware in your pocket. In: NDSS, pp. 1-15 (2014)
3. Felt, A.P., Chin, E., Hanna, S., Song, D., Wagner, D.: Android permissions demystified. In: Proc. 18th ACM CCS, pp. 627-638 (2011)
4. Chin, E., Felt, A.P., Greenwood, K., Wagner, D.: Analyzing inter-application communication in Android. In: Proc. 9th ACM MobiSys, pp. 239-252 (2011)
5. Li, L., Bissyande, T.F., Papadakis, M., et al.: Static analysis of Android apps: A systematic literature review. Inf. Softw. Technol. 88, 67-95 (2017)
6. Octeau, D., McDaniel, P., Jha, S., et al.: Effective inter-component communication mapping in Android. In: IEEE S&P, pp. 500-514 (2013)
7. Grace, M.C., Zhou, Y., Wang, Z., Jiang, X.: Systematic detection of capability leaks in stock smartphones. In: NDSS, pp. 1-14 (2012)
8. Chen, S., Neasbitt, M., Li, Z.: Adaptive Android malware detection based on bytecode analysis. IEEE Trans. Inf. Forensics Secur. 14(8), 2110-2124 (2019)
9. Mariconti, E., Onwuzurike, L., Andriotis, P., et al.: MaMaDroid: Detecting Android malware by building Markov chains. In: NDSS, pp. 1-15 (2017)

10. Bhoraskar, R., Han, S., Jeon, J., et al.: Brahma: Component exposure analysis in mobile applications. IEEE Trans. Mob. Comput. 19(5), 1142-1156 (2020)
11. Rasthofer, S., Arzt, S., Bodden, E.: Classifying and categorizing Android application permissions. In: ESORICS, LNCS, vol. 8712, pp. 396-414. Springer (2014)
12. Arzt, S., Rasthofer, S., Fritz, C., et al.: FlowDroid: Precise context, flow, field taint analysis for Android. ACM SIGPLAN Not. 49(6), 259-269 (2014)
13. Wei, F., Roy, S., Ou, X.: Amandroid: An inter-component data flow analysis framework. ACM Trans. Priv. Secur. 21(3), 1-32 (2018)
14. Wang, Y., Zheng, J., Sun, C., Miu, S.: Quantitative risk evaluation models for mobile operating system capabilities. In: ICICS, LNCS, vol. 11149, pp. 215-230. Springer (2018)
15. Benoit, M., Tremblay, G., Gagnon, F.: On-device static analysis of mobile binaries for privacy enforcement. In: SecureComm, LNICST, vol. 398, pp. 142-161. Springer (2021)